

BlazeSync — Full Technical Specification

Frontend + Backend, everything built and planned, post-pivot (virtual accounts, roster, Course Advisor, AI layer)

This supersedes earlier technical drafts. Pairs with the Master PRD (product/business source of truth) — this is the implementation source of truth.

1. Frontend — Everything to Build

1.1 Shared / Public

- Landing page
- Auth: Sign up, Log in, Forgot password, Email/phone verification
- Role detection on login (Treasurer / Exco / Member / Course Advisor) — mostly driven by invite link, with manual fallback

1.2 Treasurer / Exco Portal

- **Association setup wizard** — name, institution, department/faculty, link treasury account via Ecobank consent flow
- **Invite exco co-signatories** — email/phone, role assignment
- **Roster upload** — CSV/Excel import of expected members (name, matric no, email/phone)
- **Virtual account provisioning** — triggered automatically per roster entry once uploaded; shows provisioning status (pending/active) per member
- **Roster dashboard** — claim status (joined app or not) + payment status per active dues cycle, independent of each other
- **Manual payment entry** — mark a roster member paid outside the app (cash/manual transfer), keeps ledger authoritative during rollout
- **Dues cycle manager** — title, amount, deadline, per-level breakdown (optional), and the **Expectation Statement** field (what this dues cycle funds — shown to students before/at payment)
- **Live ledger dashboard** — running balance (with reconciliation status), inflow/outflow feed, filters
- **Disbursement request form** — amount, reason/category, recipient
- **Approval queue** — pending disbursements awaiting co-signatory action
- **Ask BlazeSync (exco view)** — natural-language query bar over the association's own ledger ("how much did we spend on printing this semester?")
- **Audit/export view** — downloadable ledger history for handover

- **Settings** — approval threshold config, notification preferences, (Vision) Course Advisor mandate toggle

1.3 Member Portal

- **Home/dashboard** — active dues cycles across all associations the member belongs to
- **Claim invite flow** — single-use code/link → sign up or log in → linked to roster record + their provisioned virtual account
- **Pay dues flow** — shows the **Expectation Statement** before payment ("this dues cycle funds: Dept Dinner, exam prep sessions, welfare support"), then displays the member's unique virtual account details to pay into
- **Receipt view** — hashed receipt + the Expectation Statement attached permanently, "Verify" state showing the hash recomputation match
- **Shared ledger view (read-only)** — same live balance/feed exco sees
- **Ask BlazeSync (member view)** — same natural-language query bar, scoped read-only
- **Payment history** — personal record across semesters/associations

1.4 Course Advisor Portal — Vision, not MVP

- Disbursement voucher review with document previews
- Line-item approve/flag/reject
- Digital signature execution for fund release
- Full compliance audit package export (*Build as static/mockred screens only if there's spare time near the end — never let this compete for hours against the MVP golden path.*)

1.5 Cross-cutting frontend concerns

- Real-time ledger updates (WebSocket, with a visible "live" indicator)
- Useful empty states, explainable error states
- Fully responsive, phone-first
- Accessible: keyboard nav, focus states, contrast

2. Backend — Everything to Build

2.1 Core services/modules

- **Auth service** — registration, login, JWT issuance + refresh, RBAC (treasurer / exco / member / advisor)
- **Association service** — CRUD, membership, role assignment
- **Roster service** — bulk import, virtual account provisioning trigger, invite code generation + mass send, claim handling, manual payment marking

- **Virtual account service** — wraps Ecobank's virtual account issuance; one account per `MemberRecord`, maps incoming payments back to the owning member automatically
- **Dues service** — dues cycle CRUD (including Expectation Statement field), tracks who owes/has paid against the roster
- **Payment service** — receives webhook-confirmed virtual account credit, triggers receipt + ledger entry (no manual "pay" orchestration needed the way a shared Collection account required — the virtual account *is* the attribution mechanism)
- **Ledger service** — append-only entries, running balance
- **Reconciliation service** — periodic + on-demand comparison of our ledger balance vs. Ecobank's Account Enquiry balance for the treasury account
- **Receipt service** — hash generation + recomputation/verification, attaches Expectation Statement snapshot at time of payment (so later edits to a dues cycle's statement don't retroactively change old receipts)
- **Disbursement service** — request creation, approval state tracking, triggers Local Bank Payment once threshold met
- **Approval/multi-sig service** — records distinct exco approvals, enforces threshold server-side
- **Ask BlazeSync service (AI, MVP)** — see Section 6
- **BlazeSync Sentinel service (AI, Vision)** — see Section 6
- **Ecobank integration layer** — token generation, secureHash computation, virtual account issuance, Account Enquiry, Local Bank Payment, Notification/webhook handling
- **Webhook handler** — verifies source, routes virtual-account-credit events to Payment service, routes other events appropriately
- **Notification service (internal)** — invite sent, payment confirmed, disbursement pending/completed
- **Audit log service** — immutable action log (who did what, when) — separate from the financial ledger

2.2 Course Advisor approval step — Vision, not MVP

- `AdvisorApproval` model + endpoint, inserted between exco approval and fund release in the disbursement flow
- Document preview/upload handling (would need object storage — S3 or equivalent)
- Digital signature capture

2.3 Supporting infrastructure

- Background jobs (FastAPI BackgroundTasks for MVP) — webhook processing, mass invites, reconciliation checks, AI query handling if slow

- Rate limiting
 - Idempotency keys on payment/disbursement-affecting endpoints
-

3. Data Models (Full, Post-Pivot)

User

- id, name, email, phone, password_hash, created_at

Association

- id, name, institution, department_or_faculty, treasury_account_ref, created_by, approval_threshold, advisor_mandate_enabled (bool, vision flag), created_at

Membership

- id, user_id, association_id, role (treasurer | exco | member | advisor), joined_at

MemberRecord # roster – source of truth for who owes dues

- id, association_id, name, matric_number, email, phone, user_id (nullable until claimed), invite_code, invite_status, virtual_account_ref (nullable until provisioned), created_at

DuesCycle

- id, association_id, title, amount, deadline, status, expectation_statement (text – what this cycle funds), created_by, created_at

Payment

- id, dues_cycle_id, member_record_id, amount, ecobank_transaction_ref, status, idempotency_key, timestamp

Receipt

- id, payment_id, hash, expectation_statement_snapshot, generated_at

LedgerEntry # APPEND-ONLY

- id, association_id, type (inflow | outflow), amount, reason_or_category, linked_payment_id, linked_disbursement_id, running_balance, created_at

DisbursementRequest

- id, association_id, requested_by, amount, reason, recipient_details, status (pending | approved | rejected | advisor_review | completed), ecobank_transaction_ref, idempotency_key, created_at

DisbursementApproval

- id, disbursement_id, approved_by, decision, timestamp
- UNIQUE(disbursement_id, approved_by)

AdvisorApproval # Vision

- id, disbursement_id, advisor_id, decision, signature_ref, timestamp

AuditLog

- id, actor_id (nullable), action, metadata (JSON), timestamp

4. API Endpoints (Full, Post-Pivot)

Auth

POST /auth/register · POST /auth/login · POST /auth/refresh · POST /auth/logout

Associations & Roster

POST /associations · POST /associations/{id}/link-account · POST /associations/{id}/invite-exco · POST /associations/{id}/roster/upload · POST /associations/{id}/roster/provision-accounts (triggers virtual account creation per uploaded member) · POST /associations/{id}/roster/send-invites · GET /associations/{id}/roster · POST /roster/claim · POST /associations/{id}/roster/{member_record_id}/mark-paid

Dues Cycles & Payments

POST /associations/{id}/dues-cycles (includes expectation_statement) · GET /associations/{id}/dues-cycles · PATCH /dues-cycles/{id} · GET /payments/{id}/receipt (returns receipt + verified: true/false from hash recomputation) · GET /users/me/payments

Ledger

GET /associations/{id}/ledger · GET /associations/{id}/ledger/live (WebSocket) · GET /associations/{id}/balance/reconcile

Disbursements

POST /associations/{id}/disbursements · GET /associations/{id}/disbursements · POST /disbursements/{id}/approve · POST /disbursements/{id}/reject (Vision) · POST /disbursements/{id}/advisor-review

AI

POST /associations/{id}/ask — body: natural-language question → Ask BlazeSync (MVP) (Vision) internal Sentinel job, no direct user-facing endpoint needed — it pushes notifications when it flags something

Webhooks

`POST /webhooks/ecobank/notification` — handles virtual account credit events (routes to Payment service) and any other Ecobank event types, verifies signature first

5. Tech Stack

Layer	Choice
Backend	FastAPI (Python)
Database	PostgreSQL
ORM	SQLModel
Cache/queue	Redis (if time allows); FastAPI BackgroundTasks is the MVP default
Auth	JWT (access + refresh), bcrypt/argon2
Frontend	Next.js (React) + Tailwind
Real-time	WebSockets (FastAPI native)
AI	LLM API call (your choice of provider) with ledger data fetched and injected as context — see Section 6
Deployment	Railway/Render (backend), Vercel (frontend)

6. AI Implementation Detail

Ask BlazeSync — MVP, build this

Approach: retrieval, not fine-tuning. On a question, the backend:

- Fetches the relevant scoped data — the association's `LedgerEntry` rows (filtered by date range if the question implies one), the active `DuesCycle` and its `expectation_statement`, and roster payment status if the question is about who's paid
- Builds a compact, structured context block (not the entire ledger dumped raw — summarize/aggregate where sensible, e.g. totals by category)
- Sends the question + context to an LLM with a system prompt constraining it to answer **only from the provided data**, and to say so plainly if the data doesn't contain the answer (never let it guess or hallucinate a number)
- Returns the answer to the frontend chat UI

This is explainable to a judge in one sentence: *"It's not predicting anything — it's reading the real ledger data and answering in plain language instead of making someone scroll a transaction feed."*

BlazeSync Sentinel — Vision, do not attempt to build for the hackathon

Documented in the Master PRD as the roadmap answer to fraud detection (anomaly scoring on disbursement patterns — size relative to history, structuring detection, pre-handover spikes). State it as architecture-ready, not built, if asked.

7. Security & Auth (Unchanged Principles, Reconfirmed Post-Pivot)

- Passwords: bcrypt/argon2, never reversible
- JWT access (short-lived) + refresh (revocable) tokens
- RBAC enforced server-side on every endpoint, scoped to `association_id`
- Multi-sig threshold enforced at the database/service layer, never just the UI
- Idempotency keys on payment and disbursement endpoints
- Ledger is append-only — no UPDATE/DELETE exposed at the API layer, ever
- Receipt hash is a deterministic function of (member_record_id, amount, timestamp, association_id, ecobank_transaction_ref) — independently recomputable
- Virtual account linkage is scoped per member — one account, one owner, no shared attribution ambiguity (this is actually a security *improvement* over the old shared-collection-account model, since there's no reconciliation guesswork at all)
- Account-linking consent (treasury account) is explicit, scoped, revocable
- Webhook signature verified before any ledger-affecting action is taken
- Ask BlazeSync is read-only and scoped to the requesting user's associations — it cannot trigger any state-changing action, only answer questions from already-authorized data
- Immutable audit log, separate from the financial ledger

One-line answer if asked "how do we know the AI isn't a risk": *"It's read-only, it only sees data the user is already authorized to see, and it's explicitly instructed to say 'I don't know' rather than guess — it can't move money or change the ledger, it can only explain it."*

8. What's Actually Different From the Pre-Pivot Spec

For the team's own clarity, here's exactly what changed and why:

- **Collection Service (shared account) → Virtual accounts per member:** removes manual/ambiguous reconciliation entirely, doesn't just automate it
- **Roster is now tied to virtual account provisioning,** not just invite tracking — uploading the roster is what triggers account creation, not a separate step

- **Added Expectation Statement** to **DuesCycle** and **Receipt** — directly answers the "we don't see the benefit" complaint from survey data, passively, without requiring ledger engagement
 - **Added Course Advisor as a Vision-only role** — not in the MVP build, but modeled in the data schema so it's a believable near-term extension, not an afterthought bolted onto the pitch
 - **Added Ask BlazeSync as the one real AI feature** — Sentinel stays firmly in Vision, never claimed as built
-

9. Build Priority (If Time Runs Short, Cut in This Order — Last Item First)

1. Course Advisor portal (never build — Vision only, mentioned not demoed)
2. BlazeSync Sentinel (never build — Vision only, mentioned not demoed)
3. Manual payment entry / roster "mark paid" admin tool (nice-to-have, not core)
4. Ask BlazeSync (build if time allows — strong demo value, but the ledger core must work first)
5. Multi-sig disbursement flow (core — must work)
6. Virtual account payment → auto-reconcile → receipt → live ledger (the absolute core — this must work perfectly, everything else is secondary)